Programming Assignment: Unit 5

Hset Paing Htoo

Introduction to Computer Science Program, University of the People

CS 1102-01 Programming 1 AY2026-T5

Instructor : Sirajo Musa (Instructor)

July 23, 2026

1. Purpose

This project implements a Course Enrollment and Grade Management System for a university, as specified in the CS 1102 Programming Assignment 5 instructions. It demonstrates:

- Encapsulation — all class fields are private, accessed only through public getters/setters.
- Instance methods that manipulate an individual object's state (enrolling a student, recording a grade).
- Static variables and methods that track information shared across every instance of a class (total enrollment count, the master course/student lists).
- A working, interactive administrator command-line interface with input validation and error handling.

2. System Design

The system is split into four classes, each with a single clear responsibility:

- Student — represents one student: their name, ID, the courses they're enrolled in, and their grades. All behavior that changes a student's own data (enrolling, recording a grade) lives here as instance methods, because each student needs their own independent copy of this data.
- Course — represents one course: its code, name, and capacity. Also holds the static totalEnrolledStudents counter, since that's a single number that describes the whole system, not any one course.
- CourseManagement — the static “hub” of the system. It holds the master list of all courses and students (as private static variables, since there's only ever one catalog for the whole university) and exposes static methods that the administrator interface calls.

- Main — the administrator-facing command-line interface. Reads menu choices, validates input, and calls into CourseManagement to do the actual work.

The key design decision the assignment is testing is: when should a variable/method be static, and when should it be an instance member? The rule this project follows throughout is — static is for information that belongs to the system/class as a whole (the course catalog, the running total of enrollments); instance is for information that belongs to one specific object (one student's grades, one course's own enrollment count).

3. How static members track information across instances

Course.totalEnrolledStudents is a private static int. Unlike courseCode or currentEnrollment (instance variables — every Course object gets its own separate copy), there is only ONE copy of totalEnrolledStudents in memory, shared by every Course object that has ever been created. Every time incrementEnrollment() succeeds on any course, that single shared counter goes up. Course.getTotalEnrolledStudents() is a static method so it can be called as Course.getTotalEnrolledStudents() without needing any particular Course object — it reports on the class as a whole.

CourseManagement.courses and CourseManagement.students work the same way: they are static because the administrator interface needs one single shared catalog of courses and one single shared roster of students for the whole running program, not a separate catalog per object.

4. Requirements-to-Implementation Map

Requirement	Where it's implemented
Private instance variables + getters/setters (Student)	Student.java – name, id, enrolledCourses, grades are all private; public getName/setName/getId/setId provided
enrollStudent method adding courses	Student.enrollInCourse(Course) (instance method) called via CourseManagement.enrollStudent(Student, Course)
assignGrade method updating a student's grade	Student.assignGrade(Course, double) (instance method) called via CourseManagement.assignGrade(Student, Course, double)
Private instance variables + getters (Course)	Course.java – courseCode, name, maxCapacity, currentEnrollment are private; public getters provided
Static variable tracking total enrolled students	Course.totalEnrolledStudents (private static int), incremented in incrementEnrollment()
Static method to retrieve total enrolled students	Course.getTotalEnrolledStudents() (public static)
Static list of courses / overall grades (CourseManagement)	CourseManagement.courses (private static List<Course>) and CourseManagement.overallGrades (private static Map<String, Double>)
addCourse / enrollStudent / assignGrade / calculateOverallGrade	All implemented as public static methods on CourseManagement, each delegating to Student/Course instance methods
Administrator command-line interface	Main.java – menu loop with options 0–7, input prompts, and validation
Error handling (invalid input, course at capacity)	Try/catch around Integer.parseInt/Double.parseDouble, explicit isFull() capacity check before enrollment, duplicate-ID/duplicate-course checks

5. Source Code

5.1 Student.java

Encapsulates a single student's data and the two behaviors the assignment requires: enrolling in a course and being assigned a grade.

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Represents a single student. All fields are private (encapsulated);
 * outside classes may only read or change them through the public
 * getters/setters and the enrollInCourse/assignGrade behavior methods
 * defined here.
 */
public class Student {

    private String name;
    private String id;
    private List<Course> enrolledCourses;
    private Map<String, Double> grades; // key = course code, value = grade

    public Student(String name, String id) {
        this.name = name;
        this.id = id;
        this.enrolledCourses = new ArrayList<>();
        this.grades = new HashMap<>();
    }

    // ---- Getters ----
    public String getName() {
        return name;
    }

    public String getId() {
        return id;
    }

    public List<Course> getEnrolledCourses() {
        return enrolledCourses;
    }

    public Map<String, Double> getGrades() {
        return grades;
    }

    // ---- Setters ----
    public void setName(String name) {
        this.name = name;
    }

    public void setId(String id) {
        this.id = id;
    }

    /**
     * Instance method: adds a course to THIS student's own list of
     * enrolled courses. Operates on this object's state, so it must
     * be an instance method, not static.
```

```
     *
     * @return true if the course was added, false if the student was
     *         already enrolled in it.
     */
    public boolean enrollInCourse(Course course) {
        if (enrolledCourses.contains(course)) {
            return false;
        }
        enrolledCourses.add(course);
        return true;
    }

    /**
     * Instance method: records a grade for this student in a course,
     * but only if the student is actually enrolled in that course.
     *
     * @return true if the grade was recorded, false if the student is
     *         not enrolled in the given course.
     */
    public boolean assignGrade(Course course, double grade) {
        if (!enrolledCourses.contains(course)) {
            return false;
        }
        grades.put(course.getCourseCode(), grade);
        return true;
    }

    public Double getGrade(Course course) {
        return grades.get(course.getCourseCode());
    }

    @Override
    public String toString() {
        return "[" + id + "] " + name;
    }
}
```

5.2 Course.java

Encapsulates a single course's data, including the static enrollment counter described in Section 3.

```
/**
 * Represents a single course offered by the university.
 *
 * Each Course object tracks its own capacity and current enrollment,
 * while a static (class-level) variable, totalEnrolledStudents, keeps
 * a running total of how many enrollments have happened across every
 * Course object that has ever been created. That single static counter
 * is what lets the whole program answer "how many students are enrolled
 * in courses overall?" without adding up every course individually.
 */
public class Course {

    // ---- Instance variables (belong to ONE course) ----
    private String courseCode;
    private String name;
    private int maxCapacity;
    private int currentEnrollment;

    // ---- Static variable (shared by ALL Course objects) ----
    private static int totalEnrolledStudents = 0;

    /**
     * Creates a new course with zero students currently enrolled.
```

```
     */
    public Course(String courseCode, String name, int maxCapacity) {
        this.courseCode = courseCode;
        this.name = name;
        this.maxCapacity = maxCapacity;
        this.currentEnrollment = 0;
    }

    // ---- Getters (public, read-only access to private fields) ----
    public String getCourseCode() {
        return courseCode;
    }

    public String getName() {
        return name;
    }

    public int getMaxCapacity() {
        return maxCapacity;
    }

    public int getCurrentEnrollment() {
        return currentEnrollment;
    }

    /**
     * Instance method: is THIS course full? Depends on this object's
     * own currentEnrollment/maxCapacity, so it cannot be static.
     */
    public boolean isFull() {
        return currentEnrollment >= maxCapacity;
    }

    /**
     * Called by CourseManagement after a student has successfully been
     * added to this course. Bumps this course's own counter AND the
     * shared static counter.
     *
     * @return true if the enrollment count was incremented, false if
     *         the course was already full.
     */
    public boolean incrementEnrollment() {
        if (isFull()) {
            return false;
        }
        currentEnrollment++;
        totalEnrolledStudents++;
        return true;
    }

    /**
     * Static method: returns the total number of enrolled students
     * across every Course object that exists. Static because it
     * reports on the class as a whole, not on any single course.
     */
    public static int getTotalEnrolledStudents() {
        return totalEnrolledStudents;
    }

    @Override
    public String toString() {
        return courseCode + " - " + name + " (" + currentEnrollment + "/" + maxCapacity + "
enrolled)";
    }
}
```

5.3 CourseManagement.java

The static coordinator class. Holds the master static lists/maps and exposes the static addCourse / enrollStudent / assignGrade / calculateOverallGrade methods the administrator interface calls.

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Acts as the central coordinator for the whole system. It does not
 * represent any single real-world entity the way Student or Course
 * does — instead it holds the master lists of courses and students
 * and the overall grade for each student, all as private static
 * variables, and exposes static methods for the administrator
 * interface (Main) to call. Because there is only ever one "course
 * catalog" for the whole university, static state is the right choice
 * here (compare: Student/Course use INSTANCE variables because each
 * student and each course needs its own separate copy of its data).
 */
public class CourseManagement {

    private static List<Course> courses = new ArrayList<>();
    private static List<Student> students = new ArrayList<>();
    private static Map<String, Double> overallGrades = new HashMap<>(); // key = student ID

    /**
     * Creates a new Course object from the given details and adds it
     * to the master list of courses.
     */
    public static Course addCourse(String courseCode, String name, int maxCapacity) {
        Course course = new Course(courseCode, name, maxCapacity);
        courses.add(course);
        return course;
    }

    /**
     * Creates a new Student object and adds it to the master list of
     * students.
     */
    public static Student addStudent(String name, String id) {
        Student student = new Student(name, id);
        students.add(student);
        return student;
    }

    /**
     * Enrolls a student in a course by delegating to Student's own
     * instance method, then updates the course's enrollment counters.
     * Checks the course's capacity first so a full course cannot be
     * over-enrolled.
     */
    public static boolean enrollStudent(Student student, Course course) {
        if (course.isFull()) {
            return false;
        }
        boolean enrolled = student.enrollInCourse(course);
        if (enrolled) {
            course.incrementEnrollment();
        }
        return enrolled;
    }
```

```
    /**
     * Assigns a grade to a student for a course by delegating to
     * Student's own instance method, then refreshes that student's
     * overall grade.
     */
    public static boolean assignGrade(Student student, Course course, double grade) {
        boolean assigned = student.assignGrade(course, grade);
        if (assigned) {
            calculateOverallGrade(student);
        }
        return assigned;
    }

    /**
     * Calculates a student's overall grade as the average of every
     * grade they have received so far, and stores it in the static
     * overallGrades map for quick lookup later.
     */
    public static double calculateOverallGrade(Student student) {
        Map<String, Double> grades = student.getGrades();
        if (grades.isEmpty()) {
            overallGrades.put(student.getId(), 0.0);
            return 0.0;
        }
        double sum = 0;
        for (double g : grades.values()) {
            sum += g;
        }
        double average = sum / grades.size();
        overallGrades.put(student.getId(), average);
        return average;
    }

    public static List<Course> getCourses() {
        return courses;
    }

    public static List<Student> getStudents() {
        return students;
    }

    public static Course findCourseByCode(String code) {
        for (Course c : courses) {
            if (c.getCourseCode().equalsIgnoreCase(code)) {
                return c;
            }
        }
        return null;
    }

    public static Student findStudentById(String id) {
        for (Student s : students) {
            if (s.getId().equalsIgnoreCase(id)) {
                return s;
            }
        }
        return null;
    }
}
```

5.4 Main.java — Administrator Interface

A text menu loop that prompts the administrator for input, validates it, and reports success or a specific error message (invalid number, empty field, course at capacity, student/course not found, duplicate ID/code, not enrolled).

```
import java.util.List;
import java.util.Scanner;

/**
 * Administrator-facing command-line interface for the Course
 * Enrollment and Grade Management System. Reads menu choices from the
 * console, validates input, and delegates the actual work to
 * CourseManagement (which in turn delegates to Student and Course).
 */
public class Main {

    private static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        boolean running = true;

        System.out.println("=====================================================");
        System.out.println(" Course Enrollment and Grade Management System");
        System.out.println("=====================================================");

        while (running) {
            printMenu();
            String choice = scanner.nextLine().trim();

            switch (choice) {
                case "1": addCourseFlow(); break;
                case "2": addStudentFlow(); break;
                case "3": enrollStudentFlow(); break;
                case "4": assignGradeFlow(); break;
                case "5": calculateOverallGradeFlow(); break;
                case "6": viewCoursesFlow(); break;
                case "7": viewStudentsFlow(); break;
                case "0":
                    running = false;
                    System.out.println("Exiting. Goodbye!");
                    break;
                default:
                    System.out.println("Invalid option. Please enter a number from the
menu.");
            }
        }
        scanner.close();
    }

    private static void printMenu() {
        System.out.println();
        System.out.println("---------------------- MENU ----------------------");
        System.out.println("1. Add a new course");
        System.out.println("2. Add a new student");
        System.out.println("3. Enroll a student in a course");
        System.out.println("4. Assign a grade to a student");
        System.out.println("5. Calculate overall grade for a student");
        System.out.println("6. View all courses");
        System.out.println("7. View all students");
        System.out.println("0. Exit");
```

```
        System.out.print("Enter your choice: ");
    }

    private static void addCourseFlow() {
        System.out.print("Enter course code (e.g., CS101): ");
        String code = scanner.nextLine().trim();
        System.out.print("Enter course name: ");
        String name = scanner.nextLine().trim();
        System.out.print("Enter maximum capacity: ");
        String capacityInput = scanner.nextLine().trim();

        if (code.isEmpty() || name.isEmpty()) {
            System.out.println("Error: course code and name cannot be empty.");
            return;
        }
        if (findCourseSafely(code) != null) {
            System.out.println("Error: a course with code " + code + " already exists.");
            return;
        }

        int capacity;
        try {
            capacity = Integer.parseInt(capacityInput);
        } catch (NumberFormatException e) {
            System.out.println("Error: capacity must be a whole number.");
            return;
        }
        if (capacity <= 0) {
            System.out.println("Error: capacity must be greater than zero.");
            return;
        }

        Course course = CourseManagement.addCourse(code, name, capacity);
        System.out.println("Course added successfully: " + course);
    }

    private static void addStudentFlow() {
        System.out.print("Enter student name: ");
        String name = scanner.nextLine().trim();
        System.out.print("Enter student ID: ");
        String id = scanner.nextLine().trim();

        if (name.isEmpty() || id.isEmpty()) {
            System.out.println("Error: name and ID cannot be empty.");
            return;
        }
        if (CourseManagement.findStudentById(id) != null) {
            System.out.println("Error: a student with ID " + id + " already exists.");
            return;
        }

        Student student = CourseManagement.addStudent(name, id);
        System.out.println("Student added successfully: " + student);
    }

    private static void enrollStudentFlow() {
        Student student = promptForStudent();
        if (student == null) return;
        Course course = promptForCourse();
        if (course == null) return;

        if (course.isFull()) {
            System.out.println("Error: course " + course.getCourseCode()
                    + " has reached its maximum capacity (" + course.getMaxCapacity()
                    + "). Cannot enroll another student.");
            return;
        }
```

```
        boolean success = CourseManagement.enrollStudent(student, course);
        if (success) {
            System.out.println(student.getName() + " was enrolled in " +
course.getCourseCode() + ".");
        } else {
            System.out.println("Error: " + student.getName() + " is already enrolled in "
                    + course.getCourseCode() + ".");
        }
    }

    private static void assignGradeFlow() {
        Student student = promptForStudent();
        if (student == null) return;
        Course course = promptForCourse();
        if (course == null) return;

        System.out.print("Enter grade (0-100): ");
        String gradeInput = scanner.nextLine().trim();
        double grade;
        try {
            grade = Double.parseDouble(gradeInput);
        } catch (NumberFormatException e) {
            System.out.println("Error: grade must be a number.");
            return;
        }
        if (grade < 0 || grade > 100) {
            System.out.println("Error: grade must be between 0 and 100.");
            return;
        }

        boolean success = CourseManagement.assignGrade(student, course, grade);
        if (success) {
            System.out.println("Grade " + grade + " assigned to " + student.getName()
                    + " for " + course.getCourseCode() + ".");
        } else {
            System.out.println("Error: " + student.getName() + " is not enrolled in "
                    + course.getCourseCode() + ", so a grade cannot be assigned.");
        }
    }

    private static void calculateOverallGradeFlow() {
        Student student = promptForStudent();
        if (student == null) return;

        double overall = CourseManagement.calculateOverallGrade(student);
        System.out.printf("Overall grade for %s: %.2f%n", student.getName(), overall);
    }

    private static void viewCoursesFlow() {
        List<Course> courses = CourseManagement.getCourses();
        if (courses.isEmpty()) {
            System.out.println("No courses have been added yet.");
            return;
        }
        System.out.println("All courses:");
        for (Course c : courses) {
            System.out.println("  " + c);
        }
        System.out.println("Total enrollments across all courses (static counter): "
                + Course.getTotalEnrolledStudents());
    }

    private static void viewStudentsFlow() {
        List<Student> students = CourseManagement.getStudents();
        if (students.isEmpty()) {
            System.out.println("No students have been added yet.");
```

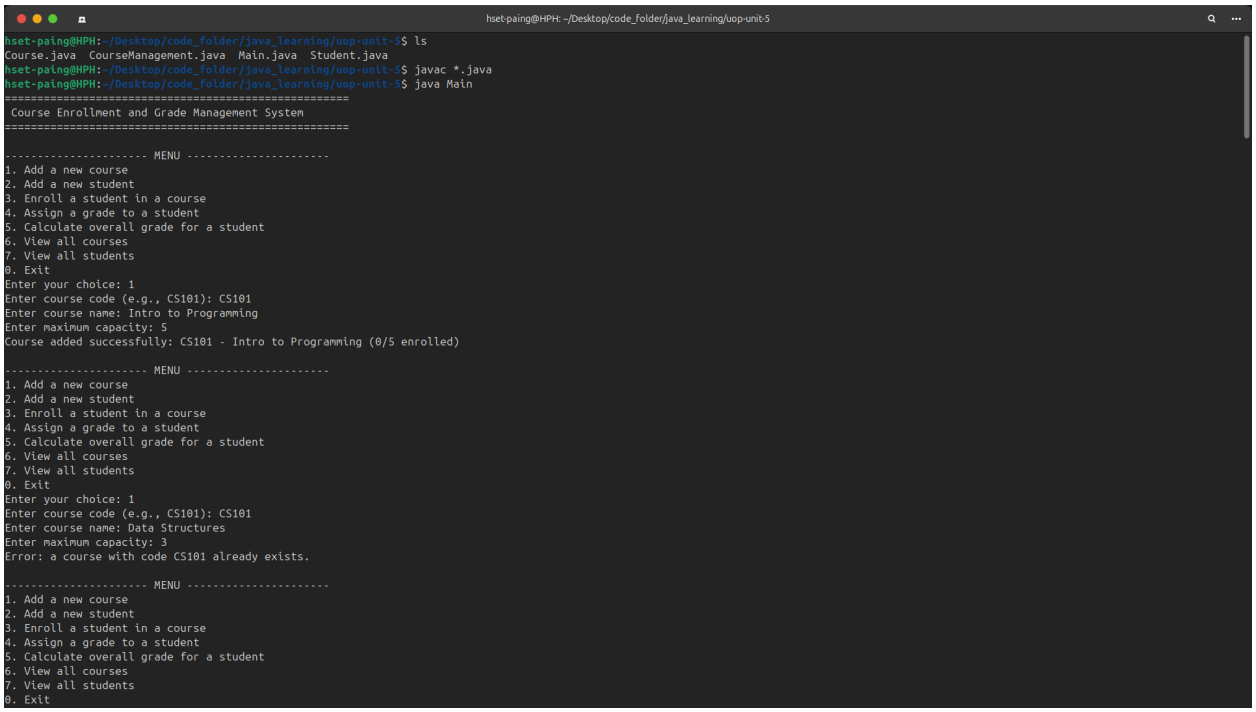
```
            return;
        }
        System.out.println("All students:");
        for (Student s : students) {
            System.out.println("  " + s + " - enrolled in " + s.getEnrolledCourses().size() +
" course(s)");
        }
    }

    // ---- Helper prompts shared by several flows ----

    private static Student promptForStudent() {
        System.out.print("Enter student ID: ");
        String id = scanner.nextLine().trim();
        Student student = CourseManagement.findStudentById(id);
        if (student == null) {
            System.out.println("Error: no student found with ID " + id + ".");
        }
        return student;
    }

    private static Course promptForCourse() {
        System.out.print("Enter course code: ");
        String code = scanner.nextLine().trim();
        Course course = CourseManagement.findCourseByCode(code);
        if (course == null) {
            System.out.println("Error: no course found with code " + code + ".");
        }
        return course;
    }

    private static Course findCourseSafely(String code) {
        return CourseManagement.findCourseByCode(code);
    }
}
```

6. How to Run the Program

- Save the four files (Student.java, Course.java, CourseManagement.java, Main.java) into the same folder.
- Compile: javac *.java
- Run: java Main
- Follow the on-screen numbered menu. Enter 0 at any time to exit.

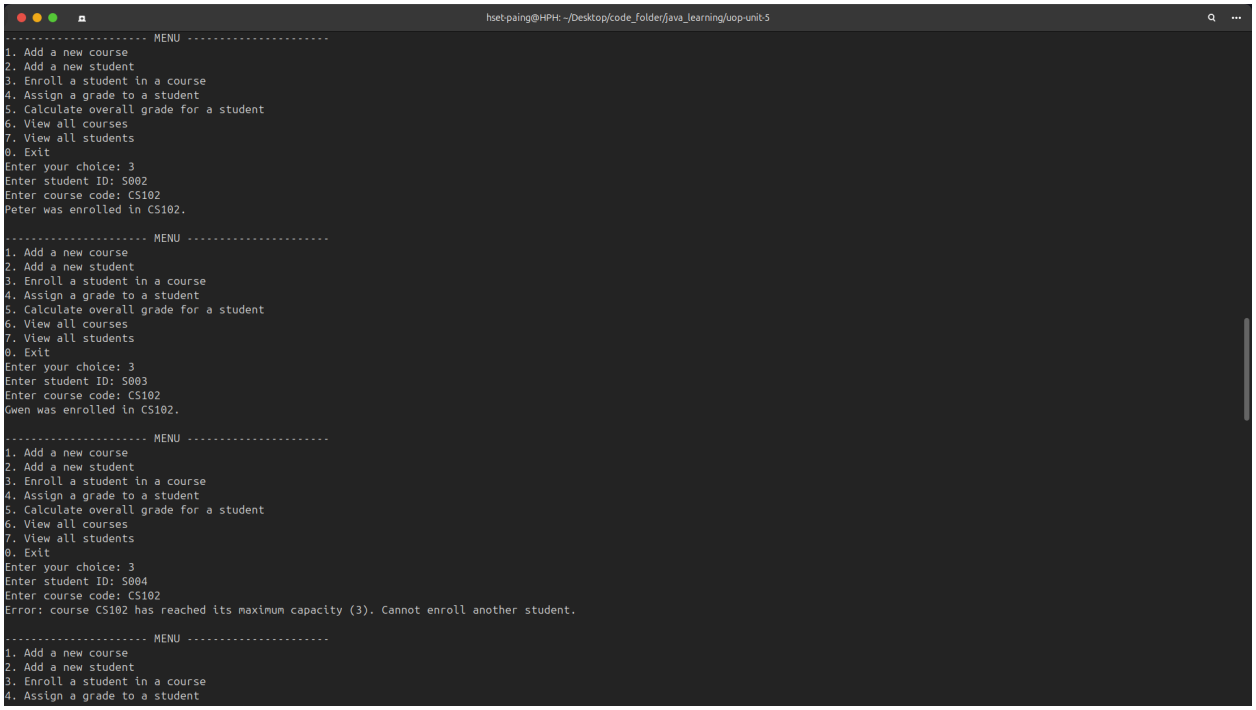
7. Sample Program Output

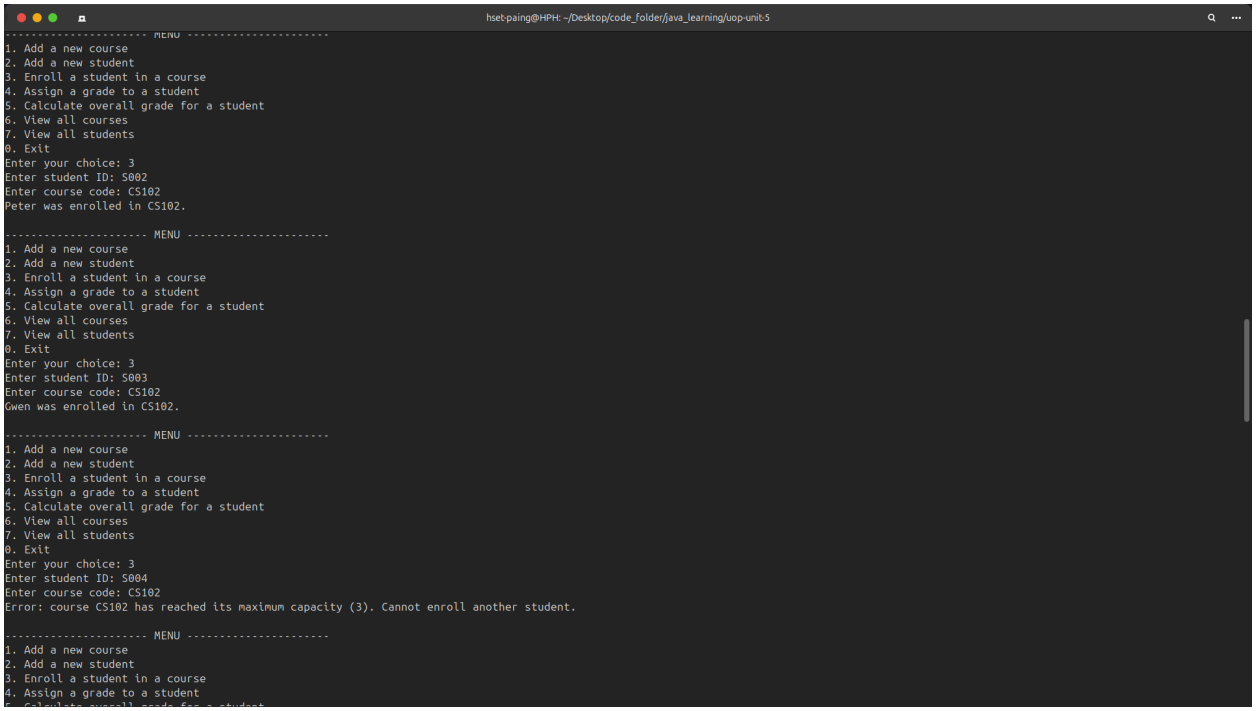
The transcript below is real console output captured from an actual run of the compiled program (javac *.java && java Main), exercising every menu option including the error-handling paths (course at capacity, invalid/empty input, duplicate IDs, grading a student who isn't enrolled). It stands in for the screenshot requested in the assignment.

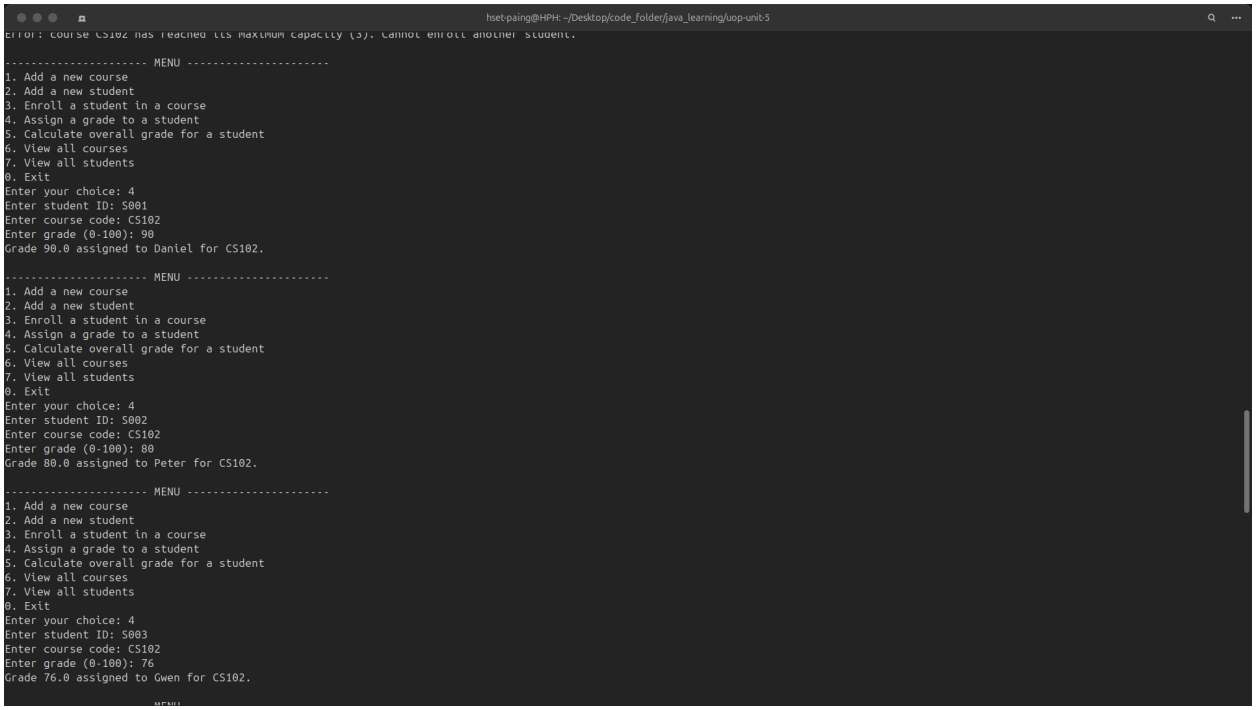


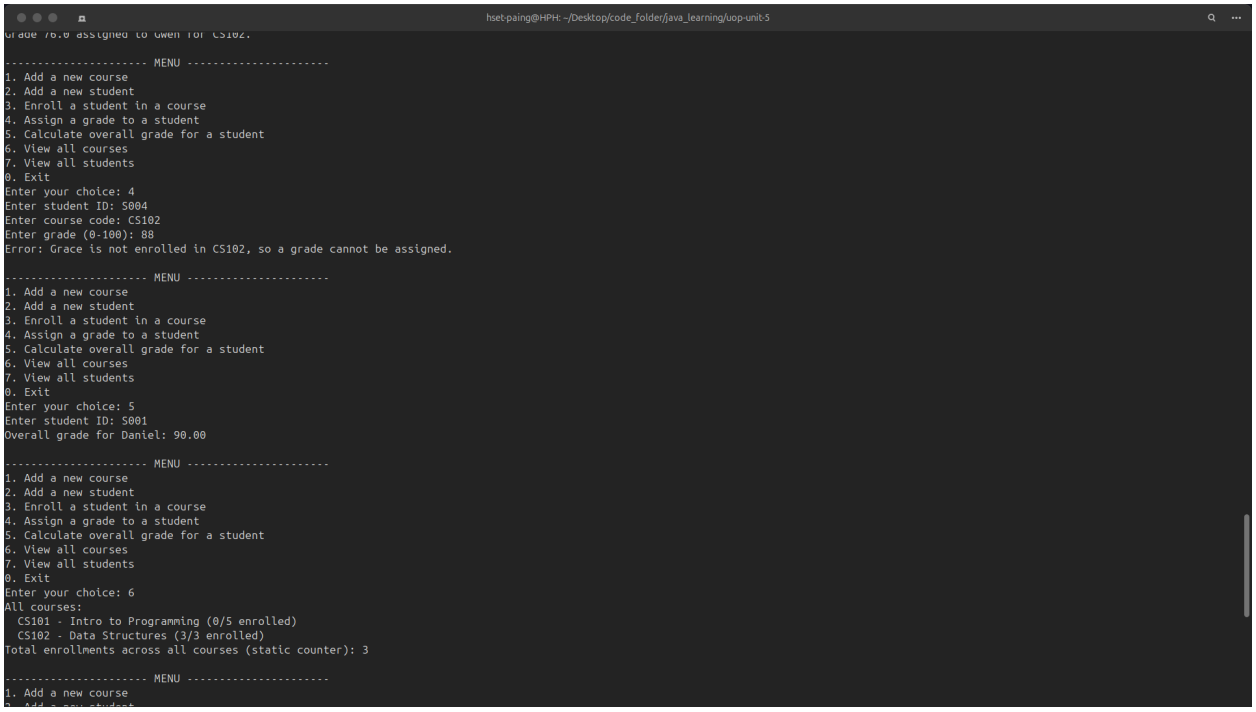
```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-5
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 1
Enter course code (e.g., CS101): CS102
Enter course name: Data Structures
Enter maximum capacity: 3
Course added successfully: CS102 - Data Structures (0/3 enrolled)

---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 2
Enter student name: Daniel
Enter student ID: S001
Student added successfully: [S001] Daniel

---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 2
Enter student name: Peter
Enter student ID: S002
Student added successfully: [S002] Peter

---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
```

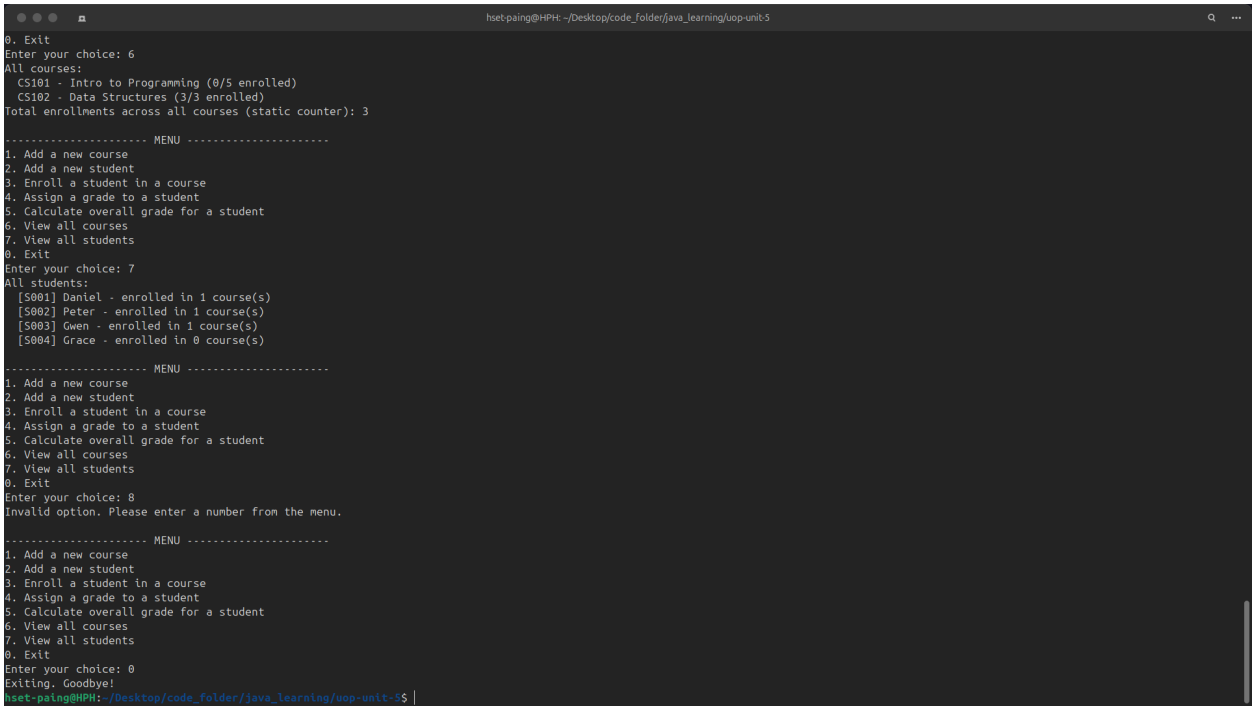
```
hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-5
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 2
Enter student name: Gwen
Enter student ID: S003
Student added successfully: [S003] Gwen

---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 2
Enter student name: Grace
Enter student ID: S004
Student added successfully: [S004] Grace

---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 3
Enter student ID: S001
Enter course code: CS102
Daniel was enrolled in CS102.

---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
```


hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-5
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 3
Enter student ID: S002
Enter course code: CS102
Peter was enrolled in CS102.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 3
Enter student ID: S003
Enter course code: CS102
Gwen was enrolled in CS102.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 3
Enter student ID: S004
Enter course code: CS102
Error: course CS102 has reached its maximum capacity (3). Cannot enroll another student.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student


hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-5
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 3
Enter student ID: S002
Enter course code: CS102
Peter was enrolled in CS102.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 3
Enter student ID: S003
Enter course code: CS102
Gwen was enrolled in CS102.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 3
Enter student ID: S004
Enter course code: CS102
Error: course CS102 has reached its maximum capacity (3). Cannot enroll another student.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student


hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-5
Error: Course CS102 has reached its maximum capacity (3). Cannot enroll another student.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 4
Enter student ID: S001
Enter course code: CS102
Enter grade (0-100): 90
Grade 90.0 assigned to Daniel for CS102.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 4
Enter student ID: S002
Enter course code: CS102
Enter grade (0-100): 80
Grade 80.0 assigned to Peter for CS102.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 4
Enter student ID: S003
Enter course code: CS102
Enter grade (0-100): 76
Grade 76.0 assigned to Gwen for CS102.


hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-5
Grade 76.0 assigned to Gwen for CS102.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 4
Enter student ID: S004
Enter course code: CS102
Enter grade (0-100): 88
Error: Grace is not enrolled in CS102, so a grade cannot be assigned.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 5
Enter student ID: S001
Overall grade for Daniel: 90.00
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 6
All courses:
CS101 - Intro to Programming (0/5 enrolled)
CS102 - Data Structures (3/3 enrolled)
Total enrollments across all courses (static counter): 3
---------- MENU ----------
1. Add a new course


hset-paing@HPH: ~/Desktop/code_folder/java_learning/uop-unit-5
0. Exit
Enter your choice: 6
All courses:
  CS101 - Intro to Programming (0/5 enrolled)
  CS102 - Data Structures (3/3 enrolled)
Total enrollments across all courses (static counter): 3
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 7
All students:
  [S001] Daniel - enrolled in 1 course(s)
  [S002] Peter - enrolled in 1 course(s)
  [S003] Gwen - enrolled in 1 course(s)
  [S004] Grace - enrolled in 0 course(s)
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 8
Invalid option. Please enter a number from the menu.
---------- MENU ----------
1. Add a new course
2. Add a new student
3. Enroll a student in a course
4. Assign a grade to a student
5. Calculate overall grade for a student
6. View all courses
7. View all students
0. Exit
Enter your choice: 0
Exiting. Goodbye!
hset-paing@HPH:~/Desktop/code_folder/java_learning/uop-unit-5$